

Privacy-First LLM-Assisted Penetration Testing: An Empirical Investigation of Cloud-Hosted and Local Deployment Architectures

Narmeen Safqat, Ammar Hassan, Sameer Ishaq, Zeeshan Hanif
Military College of Signals, National University of Sciences and Technology (NUST)
Islamabad, Pakistan
{safqat.n, ammar, info, zeeshan}@nust.edu.pk

Abstract—Large language models (LLMs) can automate much of the penetration-testing lifecycle, but their behaviour depends on how they are deployed, a factor prior evaluations have not isolated. We conduct a controlled three-way comparison of LLM deployment architecture as an independent variable: Gemini CLI (cloud), Gemma3:12B via Ollama (local conversational), and ShellGPT with LLaMA3.1:8B (local stateless). The three configurations run under identical Kali Linux conditions across the PTES lifecycle on Metasploitable2, DVWA, and HackTheBox MonitorsFour, and are scored on five quantitative metrics and eight qualitative dimensions. Gemini CLI gives the strongest analytical quality but transmits every target artefact to third-party inference; the stateless local pipeline achieves the only confirmed end-to-end exploitation at zero cost with no external data transfer; the conversational deployment shows systematic execution hallucination and increasing task refusal as context accumulates. We attribute the difference to *stateless prompt isolation*, which lets a local model emit security-sensitive commands that the same weights refuse in conversational mode. Building on this result, we present Argus-AI, an autonomous, privacy-by-construction vulnerability-assessment and patch-recommendation agent that runs a scan-validate-patch-rescan loop entirely on local hardware, and the Privacy-First LLM Pentesting Framework (PLPF) that governs deployment selection. The central finding is that deployment architecture, not model scale, governs field-deployable performance under privacy constraints, which makes locally hosted agentic assessment practical for sovereign and air-gapped environments where cloud APIs are prohibited.

Index Terms—Automated penetration testing, large language models, privacy-preserving security, autonomous security agent, guardrail amplification, PTES.

I. INTRODUCTION

Penetration testing exposes exploitable weaknesses before adversaries reach them, but a full-scope engagement takes weeks and a team of specialists, so most organisations cannot test continuously [1], [2]. Attackers do not share that constraint: the FBI reported \$12.5 billion in 2023 cybercrime losses, a 22% annual rise [3]. LLMs offer a way to narrow the gap by turning intent into commands, interpreting tool output, and drafting reports [4], yet they hallucinate procedures, invent commands, and stall on multi-step workflows [5], [7].

Prior evaluations overlook one variable: *deployment architecture*. Cloud models offer stronger orchestration but require every query and target artefact to leave the perimeter, which conflicts with the confidentiality obligations of professional engagements. Local models keep data on the host but differ

in reliability and hardware demand. No published study compares these architectures under identical, controlled conditions. Because professional testing is terminal-native (Nmap, Metasploit, SQLmap, Hydra, and Gobuster on Kali Linux [20]), we evaluate only CLI-based configurations against one problem set covering FTP backdoor exploitation, SSH brute-forcing, SQL injection, XSS, and LFI, classes that prior work identifies as most tractable for LLM automation [5], [12], [13].

The paper argues in two parts. Part I establishes empirically that locally deployed LLMs are operationally viable and isolates prompt architecture as the governing variable. Part II builds on that result to present Argus-AI, an autonomous, fully local closed-loop assessment agent, and the PLPF ruleset that validates it. The investigation precedes the proposal.

Research questions. **RQ1:** Does deployment architecture affect accuracy, reliability, and privacy across the PTES lifecycle? **RQ2:** Why does a stateless wrapper emit executable security commands when the same weights refuse in conversational mode? **RQ3:** How does hallucination differ between operator-executed and self-executed commands? **RQ4:** Can the validated mechanism be built into a fully local autonomous assessment agent for cloud-restricted environments?

Contributions. **C1:** Argus-AI, an autonomous, privacy-by-construction agent that runs a scan-validate-patch-rescan loop entirely on local hardware, for defence and Critical Information Infrastructure (CII) settings where cloud APIs are barred. **C2:** The first controlled comparison of cloud, local-conversational, and local-stateless LLM pentesting configurations across three targets and the PTES lifecycle. **C3:** Evidence that stateless prompt isolation enables reliable command generation while conversational context accumulation drives guardrail refusal. **C4:** The PLPF deployment-selection matrix and a dual quantitative-qualitative evaluation framework with explicit measurement bases.

II. BACKGROUND AND EVALUATED ARCHITECTURES

We follow the Penetration Testing Execution Standard (PTES): reconnaissance, scanning and enumeration, vulnerability identification, exploitation, post-exploitation, and reporting. The study covers the full lifecycle on Metasploitable2, four phases on DVWA (post-exploitation is out of web scope), and two on MonitorsFour.

The wrapper and the model are independent layers. The application layer (Ollama or ShellGPT) controls prompt framing, memory, and persistence; the weights carry the learned alignment. This separation is what lets prompt architecture be varied while the model is held fixed. Gemini CLI [17] is Google’s terminal-native cloud interface, with operator confirmation and runtime tool installation through `apt`; all engagement data reaches Google. For the local tier we provisioned a range of open-weight models through Ollama [18] to select representative executors: Mistral 7B-Instruct, LLaMA 3.1:8B, Qwen 2.5:7B, and Gemma 3:12B (with its `gemma3:12b-it-qat` quantised variant). Mistral 7B-Instruct and Qwen 2.5:7B produced valid commands but lost phase-to-phase context and were slow (Qwen 2.5:7B took over an hour for one enumeration on CPU). Gemma 3:12B became the conversational reference (Ollama chat, persistent history, text only), and LLaMA 3.1:8B through ShellGPT [19] `--shell` became the stateless reference, where each prompt is a single-shot request with no history. Because the stateless and the excluded chat-mode LLaMA share identical weights, any behavioural difference between them comes from prompt architecture alone.

III. RELATED WORK

LLM-assisted pentesting. Deng *et al.* introduced Pentest-GPT at USENIX Security, reporting a 228.6% task-completion gain over a GPT-3.5 baseline through three self-interacting modules [5]. Happe and Cito first showed that an LLM could drive autonomous privilege escalation against a vulnerable host [6], then generalised the result across a privilege-escalation benchmark in *Empirical Software Engineering*, where small local models proved weak on multi-step attacks [7], and later demonstrated autonomous assumed-breach testing of Active Directory networks [8]. Shen *et al.* proposed PentestAgent, a multi-agent framework with retrieval-augmented knowledge and validation stages [9]. Huang and Zhu added remediation to the loop with PenHeal, a two-stage framework that pairs pentesting with optimal patch selection [10], and Pratama *et al.* released CIPHER as a fine-tuned pentesting assistant [11]. These systems run almost entirely on cloud APIs and do not address the resulting confidentiality conflict.

CLI and self-hosted automation. Zaydi and Maleh showed that ShellGPT can guide an engagement from reconnaissance through post-exploitation [12], and Diaz-Parra evaluated cloud models on DVWA and HackTheBox with strong command accuracy, again cloud-only [13]. Systematic reviews of LLMs in cybersecurity report that most work relies on hosted frontier models and rarely evaluates local deployment [4], [14]. Gemini CLI has not been formally evaluated as a pentesting agent.

Hallucination and privacy. Ji *et al.* survey hallucination in generation and rank security among the highest-risk domains [15]. Yao *et al.* catalogue the model-inversion and data-extraction exposure inherent to cloud inference [16], which is acute in pentesting because target ranges, vulnerabilities, and credentials are contractually protected. Open-weight models

TABLE I
ATTACKER-MACHINE CONFIGURATION

Attribute	A: Gemini	B: Gemma3	C: ShellGPT
CPU	Core i5	Core i9	Core i9
RAM	6 GB	24 GB	8 GB
Model	cloud	~10 GB	~5 GB
store			
OS	Kali 2025.4	Kali 2025.4	Kali 2025.4
Network	HTTPS→Google	host-only	host-only
Tool	Gemini CLI	Ollama chat	ShellGPT --shell

such as LLaMA [21], Gemma [22], and Qwen [23] on Ollama, together with security-tuned releases such as Cisco Foundation-sec-8B [24] and RedSage [25], now make air-gapped deployment realistic. Across this literature, model capability and deployment architecture are treated as one, although identical weights behave very differently under stateless and conversational prompting.

Gap analysis. No study compares cloud, local-conversational, and local-stateless LLM configurations under identical CLI conditions; Gemini CLI is unevaluated as a pentesting agent; high-performing systems depend on cloud APIs without confronting confidentiality; hallucination analysis does not separate operator-executed from self-executed commands; and prompt architecture is never isolated as an independent variable, so no artefact converts a validated local-deployment finding into a fully offline closed-loop agent for environments where cloud transmission is barred. Argus-AI and the PLPF close this gap.

IV. METHODOLOGY AND EVALUATION

A. Environment

Three isolated physical machines were used, one per configuration (Table I); all targets ran on VMware in a host-only network. Metasploitable2 (vsftpd 2.3.4/CVE-2011-2523, OpenSSH 4.7p1, Telnet, Samba, web) supports the full lifecycle [29]; DVWA covers SQLi, XSS, CSRF, and LFI [30]; MonitorsFour provides a realistic black-box surface. Gemma3:12B was excluded on MonitorsFour because its hallucination on simpler targets rules out a valid comparison.

B. Penetration-Testing Workflow and Prompt Protocol

Every model received the same problem set. Gemini CLI and Gemma3:12B received identical natural-language prompts (for enumeration, “Perform service enumeration on 192.168.71.129 and identify all exploitable vulnerabilities with CVE references”); ShellGPT received a single-line `--shell` prompt (“Enumerate vulnerabilities and get username and password on 192.168.71.129”). No jailbreaking or custom system prompts were applied. Holding the LLaMA3.1:8B weights fixed between the stateless and chat lanes isolates prompt architecture, as shown in Fig. 1.

C. Evaluation Framework

Scoring used a 0–5 ordinal scale assigned from observation. Table II defines the five quantitative metrics; Table III defines the eight qualitative dimensions and their measurement bases.

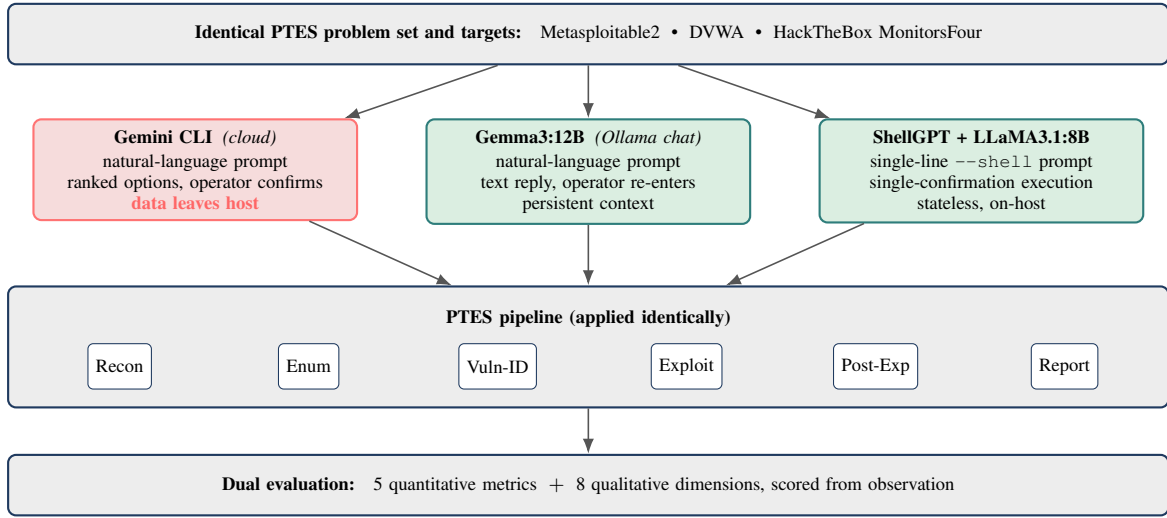


Fig. 1. Penetration-testing workflow and prompt protocol. The same problem set drives all three architectures under identical conditions; only the prompt structure and execution mode differ. Identical LLaMA3.1:8B weights in the conversational and stateless lanes isolate prompt architecture as the sole independent variable.

TABLE II
QUANTITATIVE METRICS: DEFINITION AND BASIS

Metric	Definition / measurement basis
Accuracy	Command correctness against the target; hallucinated CVEs, flags, or OS identifications lower it.
Latency	Time from prompt to actionable output; automated pipelines are weighted favourably.
Cost	Monetary cost per session; zero-cost local models score 5.
Explainability	Reasoning transparency, execution traceability, report structure.
PT Suitability	Phase completion, workflow continuity, autonomy, exploitation success.

TABLE III
QUALITATIVE DIMENSIONS: DEFINITION AND BASIS

Dimension	Definition / measurement basis
Context Retention	State preserved across prompts; cross-phase continuity.
Execution Autonomy	Unattended execution: operator re-entry versus single-confirmation run.
Output Structure	Form and machine-readability (JSON, markdown, raw logs).
Hallucination Resist.	Share of output grounded in executed results versus fabricated.
Privacy Safety	Whether data leaves the host: LOW if off-device, HIGH if on-host.
Toolchain Integ.	Ability to compose, install, and chain tools at run time.
Report Quality	Executive summary, severity classes, remediation narrative.
Guardrail Compl.	Tendency to refuse security tasks as context accumulates.

V. RESULTS AND ANALYSIS

Metasploitable2. Gemini CLI completed reconnaissance in 2:45 with JSON-structured Nmap output, confirmed CVE-2011-2523, guided exploitation transparently, and installed missing tools at run time. Gemma3:12B produced the richest prose but executes nothing; when instructed to self-execute

it identified the Linux 2.6.x host as Windows 10, reported a closed port as open, mislabelled Samba, and then confirmed that its own answer was fabricated, before refusing `/etc/passwd` extraction as security context accumulated. ShellGPT chained `nmap`, `--script=vuln`, and `Hydra` into one executable command, confirmed CVE-2011-2523 and CVE-2015-4000 through the executed scripts, recovered SSH credentials in 2,001 attempts (601 s), and established an authenticated SSH session, the only end-to-end exploitation in the study (Table IV).

DVWA. Gemini CLI mapped the SQLi, XSS, and CSRF surface, recovered from a failed tool by installing `commix`, and classified risk. Gemma3:12B explained the vulnerability classes accurately but emitted plausible payloads that do not execute against the running instance, a failure that is invisible without verification. ShellGPT analysed the source of `process-login-attempt.php` (direct SQL concatenation and an unvalidated `$_REQUEST`) and exposed the XSS vectors (Table V).

HackTheBox MonitorsFour. Gemini CLI mapped the endpoint surface with `Gobuster` and identified a critical LFI by reading a PHP warning that disclosed the internal `Router.php` path. ShellGPT ran a 65,535-port stealth scan (`nginx`, `WinRM`; OS fingerprinted as Windows Server 2022 at 88%) but, without multi-step context, did not surface the LFI (Table VI).

Aggregate. Table VII reports mean scores and Table VIII the qualitative comparison. Gemini CLI leads on accuracy, explainability, and PT suitability; ShellGPT matches it on cost and latency and is the only configuration to reach confirmed exploitation; Gemma3:12B trails because it cannot execute. Three points stand out. Context retention cuts both ways: persistence helps analysis but raises refusal, whereas stateless isolation supports reliable command generation. Guardrail compliance and hallucination resistance are independent, since Gemma3:12B is both the strictest at refusing and the most

TABLE IV
ATTACK-CHAIN COVERAGE: METASPLOITABLE2

Phase	Gemini CLI	Gemma3:12B	ShellGPT+LLaMA3.1:8B
Reconnaissance	✓ Accurate JSON output	△ Analytical description only	✓ Compound command executed
Enumeration	✓ CVE-2011-2523 confirmed	△ Based on training data	✓ CVE confirmed by executed scan
Exploitation	✓ vsftpd backdoor guided	× Self-execution hallucinated	✓ <code>user:user</code> via Hydra (601s)
Post-Exploitation	✓ Step-by-step guidance	× Refused <code>/etc/passwd</code>	✓ Authenticated SSH session
Reporting	✓ Executive summary	△ Unverified	△ Raw output

✓ verified △ partial or unverified × failed or refused

TABLE V
ATTACK-CHAIN COVERAGE: DVWA

Phase	Gemini CLI	Gemma3:12B	ShellGPT+LLaMA3.1:8B
Recon / Profiling	✓ App stack identified	✓ Descriptive overview	✓ Profiling executed
Enumeration	✓ SQLi, XSS, CSRF mapped	△ Generalised	✓ Source-code vector analysis
Exploitation	✓ DB enum; commix installed	× Hallucinated payloads	✓ SQLi and XSS via source
Reporting	✓ Risk-classified report	△ Unverified	△ Functional logs

✓ verified △ partial or unverified × failed or refused

TABLE VI
ATTACK-CHAIN COVERAGE: HACKTHEBOX MONITORSFOUR (GEMMA3:12B EXCLUDED)

Phase	Gemini CLI	ShellGPT+LLaMA3.1:8B
Reconnaissance	✓ Ports and backend via header analysis; full endpoint map	✓ 65,535-port stealth scan; nginx and WinRM; OS Windows Server 2022 (88%)
Vulnerability ID	✓ Critical LFI via <code>Router.php</code> disclosure	× Pipeline composed; LFI not identified

✓ verified × missed or failed

TABLE VII
OVERALL SCORING (MEAN ACROSS APPLICABLE TARGETS)

Metric	Gemini	Gemma3	ShellGPT
Accuracy	4.3	2.0	4.0
Latency	4.3	2.0	4.3
Cost	3.0	5.0	5.0
Explainability	5.0	3.0	3.0
PT Suitability	4.3	2.0	4.0
Total / 25	21.0	14.0	20.3
Rank	1st	3rd	2nd

Gemini and ShellGPT over 3 targets; Gemma3 over Metasploitable2 and DVWA.

prone to fabrication. Runtime toolchain integration separates the cloud agent from the local ones, because only Gemini CLI installs tools mid-engagement.

VI. ARGUS-AI: AUTONOMOUS PRIVACY-BY-CONSTRUCTION ASSESSMENT

The investigation shows that a stateless local executor can generate and run security-sensitive commands offline, while a conversational local model adds analytical depth but is unreliable as an executor. Argus-AI turns this into an autonomous, fully local agent that runs the scan–validate–patch–rescan loop used by cloud agents [26], [27], with every inference call served locally by Ollama, so engagement data never leaves the host.

A. Design Principles

(P1) *Privacy by construction*: every call targets a local endpoint, and the agent halts rather than fall back to a cloud

service. (P2) *Stateless generation*: Discovery and Re-scan use single-shot prompts, which prevents the guardrail amplification seen in Part I. (P3) *Validation against hallucination*: a finding must be reproduced by a sandboxed verification command before it can drive a patch. (P4) *Human authority*: no remediation is applied automatically, and the default authoriser denies every patch.

B. Four-Stage Closed Loop

Discovery emits one allowlisted command per PTES phase (nmap, nikto, sqlmap, searchsploit, gobuster, whatweb, enum4linux), sanitised against shell metacharacters, run locally, and reduced to a JSON finding. Validation exploits–verifies each finding in a sandbox; a judge prompt returns YES or NO, and only affirmatively judged findings with output and a clean exit code advance. Patch Proposal runs the advisor model in analytical mode and returns configuration, package, and code-level fixes for confirmed findings only. Re-scan repeats Discovery against the patched surface, and a finding is resolved once it no longer appears. The loop continues until no confirmed finding remains (Fig. 2, Alg. 1).

C. Implementation

The v0.1 prototype is a self-contained Python 3.10+ package that uses only the standard library, over seven auditable modules (`config`, `llm`, `models`, `stages`, `orchestrator`, `report`, `cli`); its only externals are a local Ollama daemon and the Kali toolchain. The executor runs at temperature 0 for determinism (reference LLaMA3.1:8B; Qwen2.5-Coder 7B and Mistral-Nemo 12B as drop-ins), and the advisor runs

TABLE VIII
QUALITATIVE OPERATIONAL COMPARISON

Dimension	Gemini CLI	Gemma3:12B	ShellGPT+LLaMA3.1:8B
Context Retention	High; persistent cloud session	High; full chat history	None; each prompt isolated
Execution Autonomy	Operator confirms each step	Operator re-enters every command	Single-confirmation autonomous run
Output Structure	Structured JSON and markdown	Rich prose; no execution trace	Verbose but functional logs
Hallucination Resist.	Strong; results traceable	Weak under self-execution	Good; grounded in executed output
Privacy Safety	LOW; data sent to Google	HIGH; on-host	HIGH; on-host
Toolchain Integ.	Strong; installs tools at run time	None	Strong; composes multi-tool chains
Report Quality	Professional; severity ratings	Detailed but unverified	Basic; functional
Guardrail Compl.	Moderate filtering	Strict; refuses rephrased tasks	Bypassed by statelessness

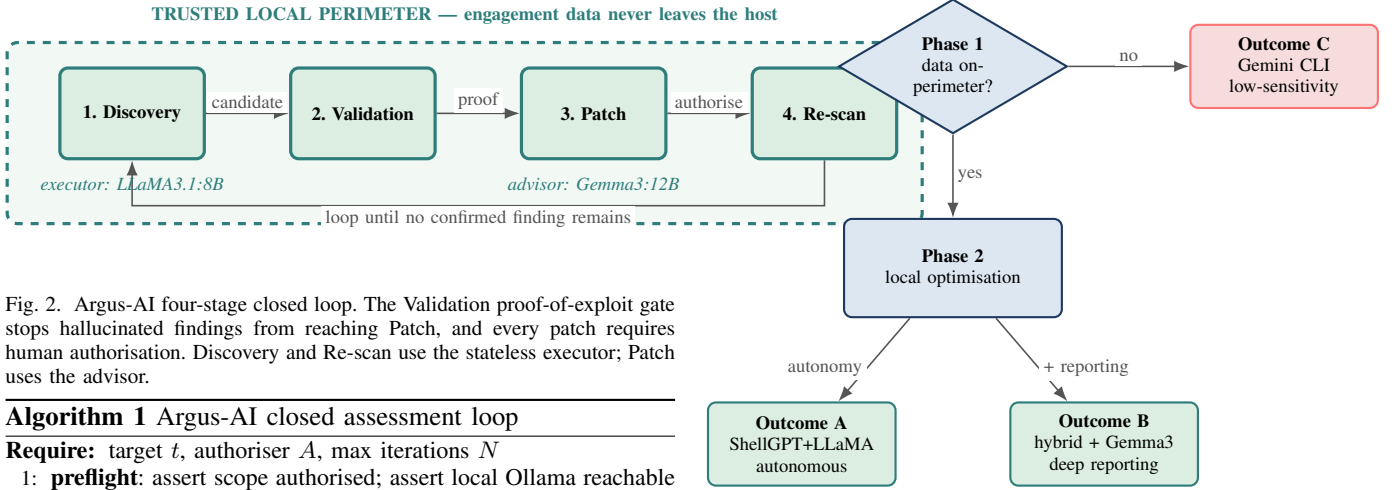


Fig. 2. Argus-AI four-stage closed loop. The Validation proof-of-exploit gate stops hallucinated findings from reaching Patch, and every patch requires human authorisation. Discovery and Re-scan use the stateless executor; Patch uses the advisor.

Algorithm 1 Argus-AI closed assessment loop

Require: target t , authoriser A , max iterations N

- 1: **preflight:** assert scope authorised; assert local Ollama reachable
- 2: **for** $i \leftarrow 1$ to N **do**
- 3: $F \leftarrow \text{DISCOVERY}(t)$; $F \leftarrow \text{VALIDATE}(F)$
- 4: $C \leftarrow \{f \in F : f.\text{verdict} = \text{CONFIRMED}\}$; $\text{PROPOSE}(C)$
- 5: **for all** $f \in C$ **do**
- 6: **if** $A(f)$ **then** apply(f)
- 7: **else** reject(f)
- 8: **end if**
- 9: **end for**
- 10: **if** patch applied **then** $F \leftarrow \text{RESCAN}(t, F)$
- 11: **end if**
- 12: **if** $\{f \in C : \neg f.\text{resolved}\} = \emptyset$ **then break**
- 13: **end if**
- 14: **end for**
- 15: **return** remediation-verified report

warmer (reference Gemma3:12B). The sandbox is a working directory with a hard timeout and a tool allowlist, hosted in a network-isolated Kali VM, and a scope gate refuses any unlisted host. Listing 1 shows the gated core of the orchestrator.

```
Listing 1. Orchestrator core: validation and human-in-the-loop patch gates.
findings = self.discovery.run(target) # stateless local discovery
findings = self.validation.run(findings) # sandbox proof-of-exploit
confirmed = [f for f in findings
              if f.verdict == Verdict.CONFIRMED]
self.patcher.run(confirmed) # advisory; confirmed only
for f in confirmed: # human-in-the-loop gate
    if cfg.require_auth and not self.authorizer(f):
        f.status = Patch.REJECTED; continue
    f.status = (Patch.APPLIED if self.apply(f) else Patch.REJECTED)
if any_applied:
```

Fig. 3. PLPF two-phase decision matrix. Privacy is resolved in Phase 1 before capability is optimised in Phase 2. Argus-AI realises Outcomes A and B.

```
findings = self.rescan.run(target, findings) # close the loop
```

VII. PLPF: THE VALIDATING RULESET

The Privacy-First LLM Pentesting Framework formalises the decision that Argus-AI embodies (Fig. 3). Phase 1 resolves the privacy boundary: if data must stay on the perimeter, every cloud model is removed from consideration. Phase 2 then optimises locally into three outcomes. Outcome A (ShellGPT+LLaMA3.1:8B) suits privacy-critical autonomous execution and covers most engagements; Outcome B (a hybrid where ShellGPT executes and Gemma3:12B reports) applies when analytical depth is also required; Outcome C (Gemini CLI) is reserved for low-sensitivity contexts. Argus-AI instantiates Outcomes A and B in one local loop, so privacy is settled before any capability trade-off.

VIII. DISCUSSION

Architecture over scale. ShellGPT with LLaMA3.1:8B outscored the larger Gemma3:12B, because stateless isolation prevents the alignment amplification that blocks the conversational model. Evaluations that equate model size with effectiveness therefore overstate what conversational models can do in autonomous execution.

Sovereignty and restricted models. For government, defence, CII, and regulated finance, data-sovereignty and classification rules such as Pakistan’s National Cyber Security Policy 2021 [28] and GDPR make cloud transmission legally incompatible with compliance. Access is also becoming fragile: frontier security-capable cloud tiers (for example the “Mythos” and “Fable” model classes) are increasingly curtailed by US Government export-control directives, which removes the strongest cloud options from the sovereign environments that most need automated assessment. Argus-AI answers both pressures. Because it treats model selection as configuration, a sovereign entity can host a specialised, unrestricted local security model such as Cisco Foundation-sec-8B [24] or RedSage [25] in the executor or advisor role, holding the architecture constant while raising capability, without giving up the privacy guarantee. We expect domain-specialised local models to be the main lever for closing the local-to-cloud analytical gap.

Reliability and target dependence. Fabricated CVEs and OS identifications are negligent in professional use, which rules out the conversational deployment as a standalone executor, and operator re-entry only reproduces the human bottleneck. Performance is also target-dependent: ShellGPT is stronger at direct tool-based exploitation, while Gemini CLI is stronger at application-layer reasoning such as LFI from a PHP error, so the choice should follow the engagement, as the PLPF prescribes.

Limitations. Lab targets do not fully reproduce production; hardware differences temper the latency comparison; single-tester scoring motivates inter-rater protocols; no security fine-tuning was applied; and Argus-AI v0.1 is single-target and bounded by its executor’s ceiling.

Ethics. All testing ran in an isolated VMware network on purpose-built or authorised targets, under the model licences and Google’s policies. Argus-AI enforces scope authorisation and human patch gates and is released for authorised use only.

IX. CONCLUSION

Across three targets and the PTES lifecycle under identical conditions, deployment architecture, rather than model scale, decided field-deployable performance. Gemini CLI led on analytical quality, the stateless local pipeline achieved the only confirmed end-to-end exploitation at zero cost with no data exposure, and the conversational deployment failed as an executor through hallucination and rising refusal. We turned this result into Argus-AI, a fully local closed-loop assessment agent, governed by the PLPF. Where confidentiality or sovereignty rule out cloud transmission, locally deployed agentic assessment is both defensible and sufficient. Future work will integrate specialised local security models into Argus-AI under single-variable comparison, add privacy-risk scoring to the PLPF, and benchmark the loop against a labelled vulnerability corpus.

REFERENCES

- [1] K. Scarfone, M. Souppaya, A. Cody, and A. Orebaugh, “Technical guide to information security testing and assessment,” NIST, Gaithersburg, MD, USA, Special Publication 800-115, 2008.
- [2] Y. Stefinko, A. Piskozub, and R. Banakh, “Manual and automated penetration testing: benefits and drawbacks. Modern tendency,” in *Proc. 13th Int. Conf. Modern Problems of Radio Engineering, Telecommunications and Computer Science (TCSET)*, IEEE, 2016, pp. 488–491.
- [3] Federal Bureau of Investigation, “2023 Internet Crime Report,” Internet Crime Complaint Center (IC3), 2024.
- [4] M. Gupta, C. Akiri, K. Aryal, E. Parker, and L. Praharaaj, “From ChatGPT to ThreatGPT: impact of generative AI in cybersecurity and privacy,” *IEEE Access*, vol. 11, pp. 80218–80245, 2023.
- [5] G. Deng *et al.*, “PentestGPT: evaluating and harnessing large language models for automated penetration testing,” in *Proc. 33rd USENIX Security Symp.*, 2024, pp. 847–864.
- [6] A. Happe and J. Cito, “Getting pwn’d by AI: penetration testing with large language models,” in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, 2023, pp. 2082–2086.
- [7] A. Happe, A. Kaplan, and J. Cito, “LLMs as hackers: autonomous Linux privilege-escalation attacks,” *Empirical Software Engineering*, vol. 31, art. 70, 2026.
- [8] A. Happe and J. Cito, “Can LLMs hack enterprise networks? Autonomous assumed-breach penetration-testing of Active Directory networks,” *ACM Trans. Softw. Eng. Methodol.*, 2025.
- [9] X. Shen *et al.*, “PentestAgent: incorporating LLM agents to automated penetration testing,” in *Proc. 20th ACM Asia Conf. Comput. Commun. Security (ASIACCS)*, 2025, pp. 375–391.
- [10] J. Huang and Q. Zhu, “PenHeal: a two-stage LLM framework for automated pentesting and optimal remediation,” in *Proc. Workshop on Autonomous Cybersecurity (AutonomousCyber)*, ACM, 2024, pp. 11–22.
- [11] D. Pratama, N. A. Suwardi, E. Ferdiansyah, R. Firmansyah, and A. Kadiptha, “CIPHER: cybersecurity intelligent penetration-testing helper for ethical researchers,” *Sensors*, vol. 24, no. 21, art. 6866, 2024.
- [12] M. Zaydi and Y. Maleh, “Empowering red teams with generative AI: transforming penetration testing through adaptive intelligence,” *ED-PACS*, vol. 70, no. 2, pp. 41–66, 2025.
- [13] R. Diaz-Parra, “Generative AI for web application penetration testing,” *Issues in Information Systems*, vol. 26, no. 1, pp. 502–514, 2025.
- [14] I. Hasanov, S. Virtanen, A. Hakkala, and J. Isoaho, “Application of large language models in cybersecurity: a systematic literature review,” *IEEE Access*, vol. 12, pp. 176751–176778, 2024.
- [15] Z. Ji *et al.*, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, art. 248, 2023.
- [16] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, “A survey on large language model (LLM) security and privacy: the good, the bad, and the ugly,” *High-Confidence Computing*, vol. 4, no. 2, art. 100211, 2024.
- [17] Google, “Gemini CLI documentation,” 2026. [Online]. Available: <https://cloud.google.com/gemini/docs/codeassist/gemini-cli>
- [18] Ollama, “Ollama: run large language models locally,” 2026. [Online]. Available: <https://ollama.com>
- [19] F. Farkhutdinov (TheR1D), “Shell-GPT: a command-line productivity tool powered by AI large language models,” GitHub, 2023. [Online]. Available: https://github.com/TheR1D/shell_gpt
- [20] Offensive Security, “Kali Linux documentation,” 2026. [Online]. Available: <https://www.kali.org/docs/>
- [21] H. Touvron *et al.*, “LLaMA: open and efficient foundation language models,” Meta AI, Tech. Rep., 2023.
- [22] Gemma Team, Google DeepMind, “Gemma: open models based on Gemini research and technology,” Tech. Rep., 2024.
- [23] Qwen Team, “Qwen2.5 technical report,” Alibaba Group, Tech. Rep., 2025.
- [24] Cisco Foundation AI, “Foundation-sec-8B: an open-weight cybersecurity language model,” Tech. Rep., 2025. [Online]. Available: <https://huggingface.co/fdn-ai/Foundation-Sec-8B>
- [25] N. Suryanto *et al.*, “RedSage: a cybersecurity generalist large language model,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2026.
- [26] OpenAI, “Aardvark: an agentic security researcher,” 2025. [Online]. Available: <https://openai.com/index/introducing-aardvark/>
- [27] Anthropic, “Claude for security research and vulnerability detection,” product documentation, 2026.

- [28] Government of Pakistan, Ministry of IT and Telecommunication, “National Cyber Security Policy 2021,” Islamabad, Pakistan, 2021.
- [29] Rapid7, “Metasploitable 2 exploitability guide,” documentation, 2012. [Online]. Available: <https://docs.rapid7.com/metasploit/metasploitable-2/>
- [30] R. Wood (digininja), “DVWA: Damn Vulnerable Web Application,” GitHub, 2012. [Online]. Available: <https://github.com/digininja/DVWA>